# ASSIGNMENT 3

TOPIC: SOFTWARE PROJECT LIFE CYCLE • SDLC MODELS & COMPARATIVE ANALYSIS

## 1. What is Project Life Cycle and its Main Characteristics?

**Definition:** A **Project Life Cycle** represents the structured sequence of phases that a software development project traverses from its initial conception and requirements elicitation to its final deployment, operational maintenance, and eventual retirement. In software engineering, it establishes an organized managerial framework to ensure software quality, on-time delivery within budgetary constraints, and verifiable alignment with institutional objectives.

**Standard Phases of a Software Project Life Cycle:**

1. *Requirement Analysis & Specification:* Capturing functional requirements (14 FAQ domains, student login, voice interaction) and non-functional requirements (security, sub-second latency, responsive UI).
2. *System & Architectural Design:* Formulating UML blueprints (Use Case, Class, Sequence), database normalization (3NF), and REST API route contracts.
3. *Implementation & Coding:* Translating designs into executable source code using Python (Flask), SQLite3, and modern frontend web standards.
4. *Integration & Testing:* Executing unit tests (test_suite.py), functional validation, and cross-browser quality checks.
5. *Deployment & Cloud Delivery:* Releasing the application into the live production cloud environment on Render with continuous delivery hooks.
6. *Operations & Maintenance:* Monitoring system uptime, updating FAQ knowledge bases, and patching dependencies.

**Main Characteristics of a Project Life Cycle:**

- **Sequential & Phased Progression:** Work is partitioned into well-defined phases where each phase possesses formal entry criteria, distinct activities, and verified exit criteria.
- **Verifiable Milestones & Deliverables:** Each phase concludes with concrete, inspectable deliverables (e.g., SRS document, UML architectural dossier, source code repository, test execution matrix).
- **Inverse Risk-to-Cost Progression:** Project risk and uncertainty are highest at the project's inception and decrease progressively as requirements solidify; conversely, the financial and temporal cost of accommodating requirement changes increases exponentially in later phases.
- **Stakeholder Governance & Traceability:** Facilitates structured oversight by mentors, academic guides, and external examiners through scheduled milestone reviews.

## 2. Explain following SDLC models.

**i) Waterfall Model (Classical Linear Sequential Model):**

The Waterfall Model is a rigid, linear software development methodology where each phase (Requirements → System Design → Implementation → Testing → Deployment → Maintenance) flows strictly downward like a waterfall. A phase cannot begin until the preceding phase is 100% complete, reviewed, and signed off.

- *Strengths:* Highly structured, simple to understand and manage; clear documentation produced at each stage.
- *Weaknesses:* Extreme inflexibility to changing requirements; working software is delivered only at the very end of the life cycle; high risk of late-stage project failure.
- *Applicability:* Suitable only for projects with frozen, crystal-clear requirements and stable technology stacks.

**ii) Iterative Model (Applied Methodology in College AI Chatbot):**

The Iterative Model develops a software system through repeated cycles (iterations), progressively expanding functionality with each release. Rather than attempting a full implementation at once, an initial simplified prototype is developed, tested, and subsequently refined based on evaluation feedback.

- *Execution in This Project:* Our project strictly followed 6 iterative release cycles:
  - *Iteration 1:* Basic text query parsing and dictionary matching in Python.
  - *Iteration 2:* Relational database schema creation (SQLite3) and Flask REST API endpoints.
  - *Iteration 3:* Student authentication module and personalized attendance percentage/detention check.
  - *Iteration 4:* Admin control center, salted password hashing, dynamic FAQ CRUD, and CSV audit export.
  - *Iteration 5:* Web Speech API voice recognition (STT), speech synthesis (TTS), Light/Dark themes, and PWA setup.
  - *Iteration 6:* Automated unit test suite execution (test_suite.py), documentation, and Render cloud deployment.
- *Strengths:* Early working software visibility; early detection of architectural flaws; easily accommodates user feedback.

**iii) Spiral Model (Risk-Driven Evolutionary SDLC):**

Proposed by Barry Boehm, the Spiral Model combines the systematic, phased aspects of the Waterfall model with the iterative prototyping of evolutionary models, governed by intensive risk assessment. The process is visualized as a spiral with multiple loops, where each 360-degree loop traverses four distinct quadrants:

1. *Objective Setting:* Identifying specific phase goals, alternative architectural approaches, and operating constraints.
2. *Risk Assessment & Resolution:* Detailed technical analysis to identify, evaluate, and resolve risks (e.g., prototyping high-risk algorithms, benchmarking server loads).
3. *Development & Validation:* Engineering, coding, and testing the software artifact for that cycle.
4. *Planning Next Phase:* Reviewing current deliverables with stakeholders and planning the next spiral cycle.

• *Strengths:* Superior risk management; ideal for mission-critical, large-scale, high-uncertainty industrial projects.
• *Weaknesses:* High project management complexity; requires specialized risk analysis expertise; expensive for small projects.

**iv) V-Shaped Model (Verification and Validation Model):**

The V-Shaped Model is an advanced extension of the Waterfall model where each development (verification) phase has a corresponding testing (validation) phase mapped opposite to it, forming a symmetrical V shape. Testing plans and test cases are authored concurrently alongside the respective design documents before coding begins:

• *Requirements Analysis* maps to • *User Acceptance Testing (UAT)*.
• *System Architectural Design* maps to • *System Testing*.
• *Module & Database Design* maps to • *Integration Testing*.
• *Detailed Component Coding (Vertex of V)* maps to • *Unit Testing*.
• *Strengths:* High defect prevention rate; test cases developed early prevent requirement ambiguities; strict quality rigor.
• *Weaknesses:* Inflexible to scope alterations once development commences; does not provide early prototypes.

**v) Agile Model (Adaptive Iterative Methodology):**

The Agile Model is an iterative, incremental development philosophy grounded in the Agile Manifesto. It delivers software in small, rapid, time-boxed increments called **sprints** (typically 1 to 4 weeks long). At the conclusion of every sprint, a potentially shippable, fully tested product increment is evaluated by stakeholders.

• *Core Values:* Individuals and interactions over processes and tools; working software over comprehensive documentation; customer collaboration over contract negotiation; responding to change over following a plan.
• *Popular Frameworks:* Scrum (roles: Scrum Master, Product Owner, Dev Team; events: Sprint Planning, Daily Standup, Sprint Review/Retrospective) and Kanban.
• *Strengths:* Maximum flexibility; immediate response to changing client demands; continuous stakeholder collaboration and early ROI.

### Comprehensive SDLC Comparison Matrix

| SDLC Model | Structural Flow | Requirement Flexibility | Risk Management | Working Software Release | Best Suited For |
|---|---|---|---|---|---|
| **Waterfall** | Linear / Sequential | Very Low (Rigid) | Poor (Late Stage) | End of Life Cycle | Fixed, frozen, well-understood specs |
| **Iterative** | Repeated Cycles | Medium to High | Moderate | Progressive Iterations | Academic & commercial web apps (Chatbot) |
| **Spiral** | Evolutionary Loops | High | Excellent (Dedicated Quadrant) | After each risk loop | Complex, high-risk enterprise systems |
| **V-Shaped** | Symmetrical V (V&V) | Low | Moderate (Defect Prevention) | Late (Post validation) | Safety-critical systems (medical, defense) |
| **Agile** | Time-boxed Sprints | Extreme (Embraces Change) | Continuous Review | At end of each 1-4 wk sprint | Fast-evolving commercial startup software |